ECA304 PROGRAMAÇÃO ORIENTADA A OBJETOS

Prof. Anderson H. Moreira

Aula 08: Classes e Objetos III

Revisando e Adicionando Algo

- Considere o código a seguir:

```
class primeiraClasse():
    """Aqui vou colocar a documentacao da minha classe.
    Ela e exibida ao se executar help(primeiraClasse)."""
    def __init__(self, parametro):
        self.parametro = parametro
    def __str__(self):
        return """Vou retornar esta string quando se utilizar o comando print em uma instância.
        \rO valor do meu parametro e {0}""".format(self.parametro)
    def __add__(self, outroParametro):
        return (primeiraClasse(self.parametro + outroParametro))

teste = primeiraClasse(1)
help(teste)
print(teste)
teste = teste + 11
print(teste)
```

- Este conceito é conhecido como **Sobrecarga de Método** e será discutido nos próximos *slides*.

Operadores Numéricos

Operador	Expressão	Método
Addition	q1 + q2	__add__(self, q2)
Subtraction	q1 – q2	__sub__(self, q2)
Multiplication	q1 * q2	__mul__(self, q2)
Power	q1 ** q2	__pow__(self, q2)
Division	q1 / q2	__truediv__(self, q2)
Floor Division	q1 // q2	__floordiv__(self, q2)
Modulo (Remainder)	q1 % q2	__mod__(self, q2)
Bitwise Left Shift	q1 << q2	__lshift__(self, q2)
Bitwise Right Shift	q1 >> q2	__rshift__(self, q2)

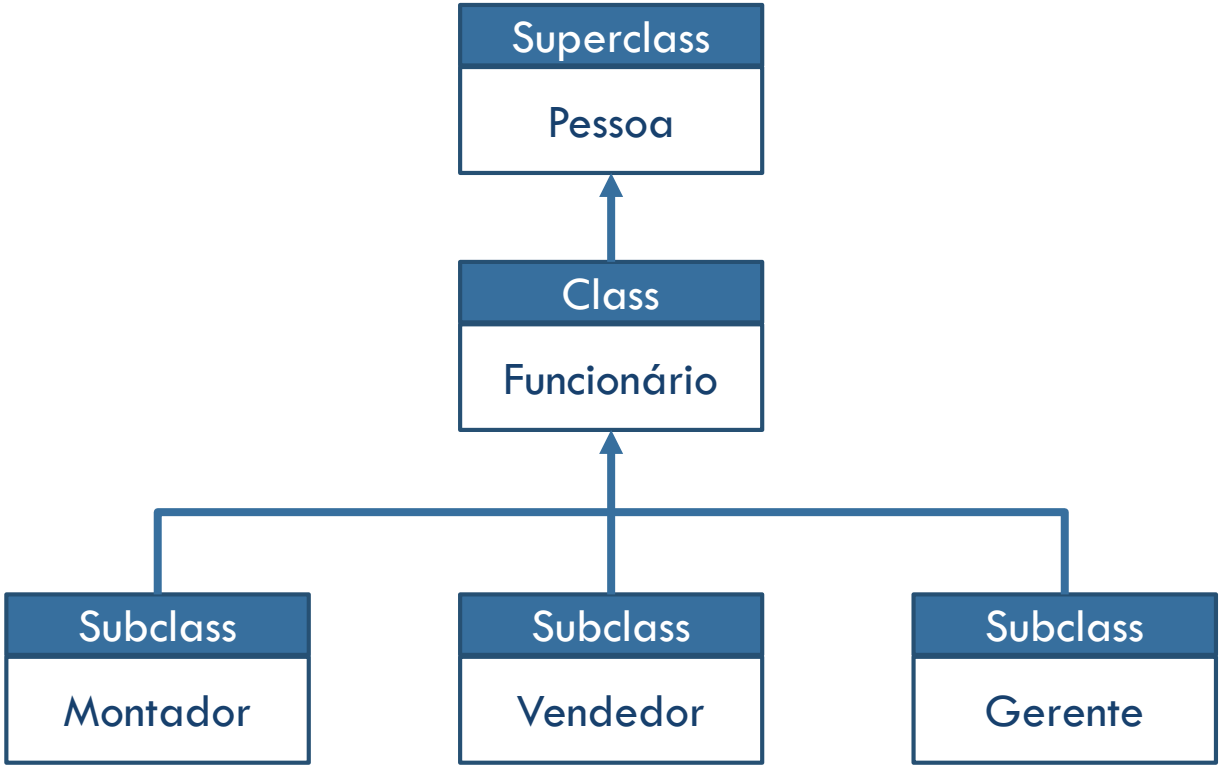
Operadores de Comparação

Operador	Expressão	Método
Less than	q1 < q2	__lt__(q1, q2)
Less than or equal to	q1 <= q2	__le__(q1, q2)
Equal to	q1 == q2	__eq__(q1, q2)
Not Equal to	q1 != q2	__ne__(q1, q2)
Greater than	q1 > q2	__gt__(q1, q2)
Greater than or equal to	q1 >= q2	__ge__(q1, q2)

Operadores Lógicos

Operador	Expressão	Método
AND	q1 & q2	__and__(q1, q2)
OR	q1 | q2	__or__(q1, q2)
XOR	q1 ^ q2	__xor__(q1, q2)
NOT	~q1	__invert__()

Vamos discutir a Herança



Vamos discutir a Herança

- Considere o código a seguir:

```
class Pessoa():
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def __str__(self):
        return 'Pessoa ' + self.nome + ' tem ' + str(self.idade) + ' anos'

class Funcionario(Pessoa):
    pass
```

Vamos discutir a Herança

- Considere o código a seguir:

```
class Pessoa():
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def __str__(self):
        return 'Pessoa ' + self.nome + ' tem ' + str(self.idade) + ' anos'

class Funcionario(Pessoa):
    def __init__(self, nome, idade, ID):
        self.nome = nome
        self.idade = idade
        self.ID = ID

    def __str__(self):
        return 'Funcionário(' + str(self.ID) + ')'
```

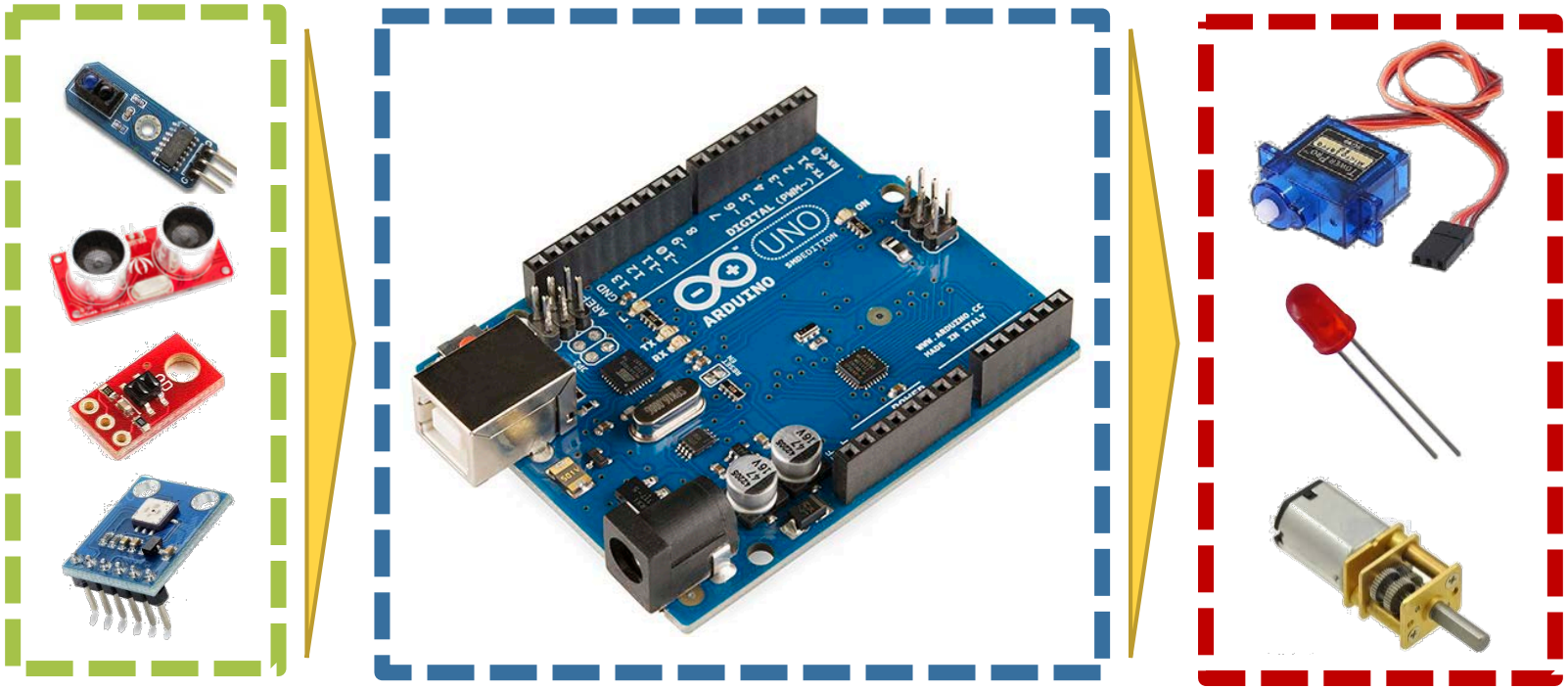
Vamos discutir a Herança

- Considere o código a seguir:

```
class Pessoa():
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def __str__(self):
        return 'Pessoa ' + self.nome + ' tem ' + str(self.idade) + ' anos'

class Funcionario(Pessoa):
    def __init__(self, nome, idade, ID):
        super().__init__(nome,idade)
        self.ID = ID

    def __str__(self):
        return super().__str__() + '-id(' + str(self.ID) + ')'
```

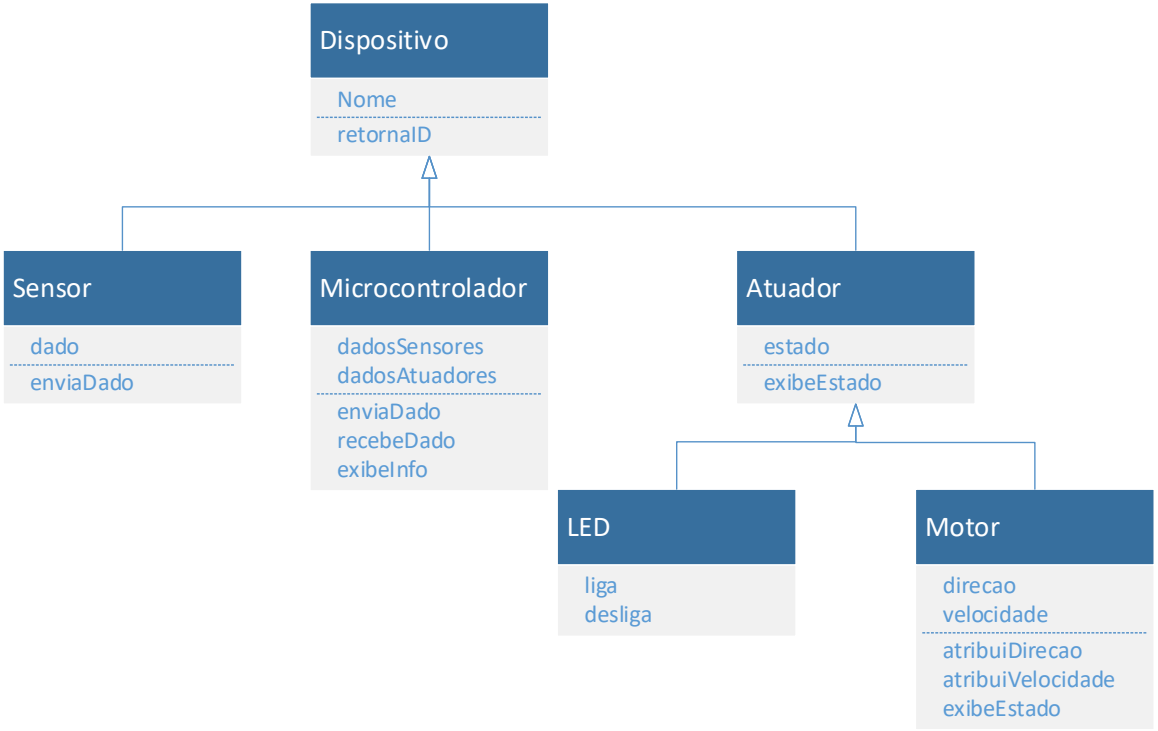
Um outro exemplo

- Consideremos o diagrama abaixo:



Um outro exemplo

- Vamos implementar as seguintes classes:



Um outro exemplo


```
class Dispositivo():
    def __init__(self, nome):
        self.nome = nome
    def retornaID(self):
        return self.nome
```

1

```
class Sensor(Dispositivo):
    def __init__(self, nome, dado = None):
        super().__init__(nome)
        self.dado = dado
    def enviaDado(self):
        from random import randrange
        self.dado = randrange(1023)
        return self.dado
```

2

```
class Microcontrolador(Dispositivo):
    def __init__(self, nome):
        super().__init__(nome)
        self.dadosSensores = dict()
        self.dadosAtuadores = dict()
    def recebeDado(self, nomeSensor, valorSensor):
        self.dadosSensores[nomeSensor] = valorSensor
    def enviaDado(self, nomeAtuador, estado):
        self.dadosAtuadores[nomeAtuador] = estado
        return self.dadosAtuadores[nomeAtuador]
    def exibeInfo(self, tipo, nomeDispositivo):
        if tipo == "sensor":
            print("{0}: {1}".format(nomeDispositivo, self.dadosSensores[nomeDispositivo]))
        if tipo == "atuador":
            print("{0}: {1}".format(nomeDispositivo, self.dadosAtuadores[nomeDispositivo]))
```

3

Um outro exemplo

- Criando dois sensores:

```
sensor1 = Sensor("Ultrassom")
arduino = Microcontrolador("Arduino")
arduino.recebeDado(sensor1.retornaID(), sensor1.enviaDado())
arduino.exibeInfo("sensor", sensor1.retornaID())

sensor2 = Sensor("Sensor de Linha")
arduino.recebeDado(sensor2.retornaID(), sensor2.enviaDado())
arduino.exibeInfo("sensor", sensor2.retornaID())
```

- E verificando o funcionamento deles:

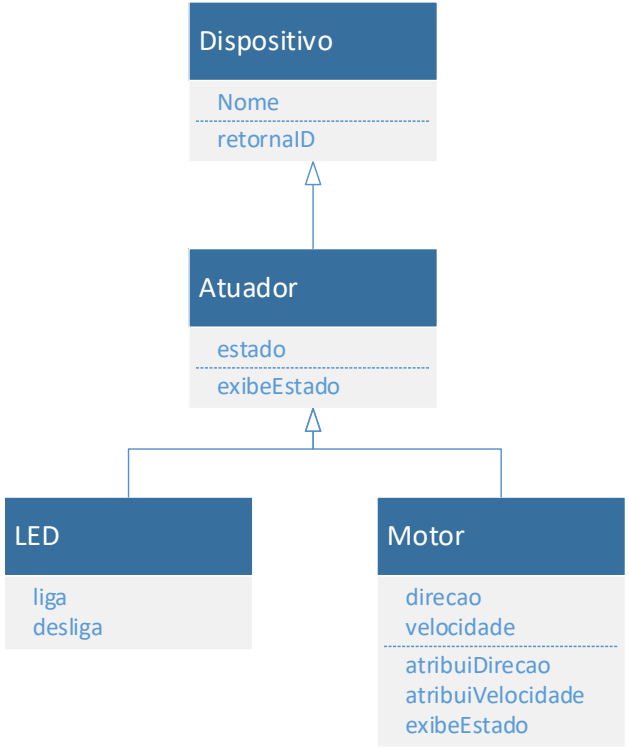
```
Ultrassom: 339
Sensor de Linha: 590
```

Exercício para entrega

Sua vez!

Implementando a classe Atuador

- Sua tarefa será implementar as seguintes classes:



Implementando a classe Atuador

- As classes **LED** e **Motor** herdam características da classe **Atuador;**
- A classe **Atuador,** por sua vez, herda características da classe **Dispositivo;**
- O atributo **estado** pode receber os valores **1 (Ligado)** e **0 (Desligado);**
- O método **exibeEstado** da classe **Atuador** imprime o **nome** do atuador e **Ligado** (se **estado = 1**) ou **Desligado** (se **estado = 0**);
- O método **liga** da classe **LED** muda o **estado** para **1;**
- O método **desliga** da classe **LED** muda o **estado** para **0;**
- O atributo **direcao** pode receber os valores **1 (Horário)** e **0 (Anti-Horário);**

Implementando a classe Atuador

- O atributo **velocidade** pode receber valores entre **0 e 255;**
- O método **atribuiDirecao** da classe **Motor** salva no atributo **direcao** o valor **1 (Horário)** ou **0 (Anti-Horário)** dependendo do parâmetro que recebe;
- O método **atribuiVelocidade** da classe **Motor** salva no atributo **velocidade** valores entre **0 e 255** dependendo do parâmetro que recebe. Se **velocidade > 0, estado = 1** senão **estado = 0;**
- O método **exibeEstado** da classe **Motor** imprime o **nome** do motor, **Ligado** (se **estado = 1**) ou **Desligado** (se **estado = 0**), caso o **estado = 1**, então deve-se imprimir também a **direcao** e **velocidade;**